

Beyond Correctness: Exposing LLM-generated Logical Flaws in Reasoning via Multi-step Automated Theorem Proving

Xinyi Zheng*
Huazhong University of
Science and Technology
Wuhan, China
xinyi_zheng@hust.edu.cn

Ningke Li*
National University of
Singapore
Singapore
ningke_li@u.nus.edu

Xiaokun Luan
Peking University
Beijing, China
luanxiaokun@pku.edu.cn

Kailong Wang†
Huazhong University of
Science and Technology
Wuhan, China
wangkl@hust.edu.cn

Ling Shi
Nanyang Technological
University
Singapore
ling.shi@ntu.edu.sg

Meng Sun
Peking University
Beijing, China
sunm@pku.edu.cn

Haoyu Wang
Huazhong University of
Science and Technology
Wuhan, China
haoyuwang@hust.edu.cn

Abstract

Large Language Models (LLMs) have demonstrated impressive reasoning capabilities, leading to their adoption in high-stakes domains such as healthcare, law, and scientific research. However, their reasoning often contains subtle logical errors masked by fluent language, posing significant risks for critical applications. While existing approaches like fact-checking, self-consistency methods, and rule-based validation provide partial solutions, they fail to detect complex logical flaws in multi-step reasoning.

To overcome these challenges, we present MATP, an evaluation framework for systematically verifying LLM reasoning via Multi-step Automatic Theorem Proving. MATP translates natural language reasoning into First-Order Logic (FOL) and applies automated theorem provers to assess step-by-step logical validity. This approach identifies hidden logical errors and provides fine-grained classifications of reasoning correctness. Evaluations on a benchmark comprising 10,830 reasoning instances generated by 10 LLMs across tasks from PrOntoQA-OOD, PROOFWRITER, and FOLIO show that MATP surpasses prompting-based baselines by over 42 percentage points in reasoning step verification. It further reveals model-level disparities, with reasoning models generating more logically coherent outputs than general models. These results demonstrate MATP's potential to enhance the trustworthiness of LLM-generated reasoning.

CCS Concepts

• **Software and its engineering** → **Software verification.**

Keywords

Large Language Model, Automated Theorem Proving, Reasoning Verification

*Both authors contributed equally to this research.

†Kailong Wang (wangkl@hust.edu.cn) is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, April 12–18, 2026, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/2026/04
<https://doi.org/10.1145/3744916.3787820>

1 Introduction

Large Language Models (LLMs) have achieved remarkable progress in natural language understanding and generation, excelling in reasoning [2, 17, 37], inference [49], and decision-making [10, 55]. Their success has driven their adoption in high-stakes domains including healthcare [6], law [21], finance [26], and scientific research [58, 60], where reliable reasoning is essential. However, safe deployment remains challenging: despite producing fluent outputs, LLMs often exhibit subtle logical errors or unsupported inferences [25, 32, 36] that could lead to serious consequences like misdiagnoses, flawed legal judgments, or erroneous research conclusions. Recent efforts to address these limitations include Large Reasoning Models (LRMs) that augment LLMs with neuro-symbolic AI, reinforcement learning, and causal inference. Yet Apple's analysis [42] reveals that even LRMs struggle with complex tasks, often relying on shallow heuristics rather than robust reasoning. Moreover, LLM reasoning remains vulnerable to adversarial attacks, where minor input perturbations can corrupt reasoning chains [11, 52].

Existing research has largely focused on detecting flawed reasoning in LLM outputs, but existing methods face key limitations. Fact-checking approaches [3, 4, 45] verify factual accuracy using external knowledge bases but often miss logical errors, especially when reasoning involves novel or implicit fact combinations. Self-consistency methods [51] reduce random mistakes via majority voting across outputs, yet fail to catch systematic errors repeated across generations. Rule-based and post-hoc validation [20, 41, 54] can flag simple contradictions but struggle with complex, multi-step logic, offering only limited coverage of reasoning flaws.

Prior works [33, 35, 57], using formal methods to enhance LLM reasoning, provide valuable insights. We consider applying formal methods to evaluate, rather than enhance, LLM reasoning as a promising direction for rigorous analysis. Despite their theoretical appeal, several challenges remain. **Challenge#1: absence of explicit logical structure in LLM-generated reasoning.** Unlike formal reasoning systems, LLMs generate reasoning via probabilistic pattern matching rather than structured inference, resulting in verbose and implicit reasoning chains. Formalizing such unstructured reasoning into precise logical representations is challenging, especially compared with Logic-LM [35], which only formalizes

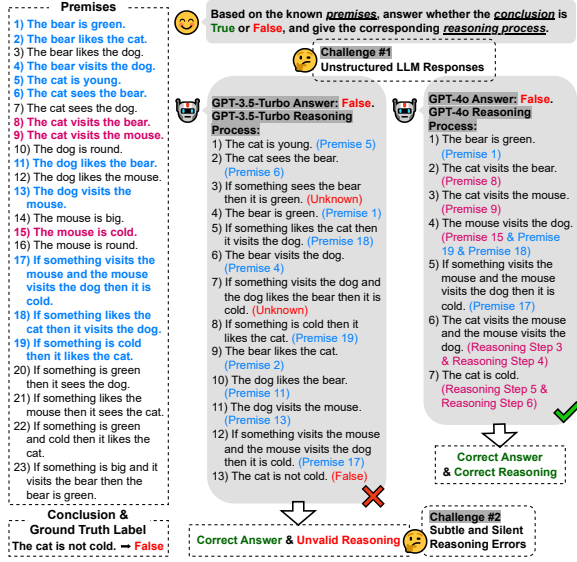


Figure 1: Motivating Example. Premises used by GPT-3.5-Turbo are marked in **Bright Blue**; GPT-4o includes these and adds **Deep Magenta** premises. GPT-3.5-Turbo’s reasoning is invalid, while GPT-4o, leveraging Premises 8, 9, and 15, yields the correct answer with a valid reasoning process.

predefined tasks. **Challenge#2: subtle and silent reasoning errors within LLMs.** A single incorrect or unsupported inference step can silently contaminate subsequent reasoning, leading to plausible but unsound conclusions. Identifying and isolating such errors is difficult because it requires precise tracking and validation of logical dependencies throughout an entire reasoning chain.

To address the challenges outlined above, we propose Multi-step Automatic Theorem Proving (MATP), a post-hoc evaluation framework that enables fine-grained analysis of LLM-generated reasoning. MATP leverages LLMs’ natural language understanding capabilities to convert reasoning tasks and model-generated chains into unified First-Order Logic (FOL) representations in a single pass (**Challenge #1**), enabling structured verification via Automated Theorem Prover (ATP). It then verifies the correctness of each reasoning step and searches for a valid proof path within the reasoning chain that logically entails the final conclusion, thereby distinguishing genuine deductive reasoning from mere information regurgitation (**Challenge #2**). Based on verification outcomes, MATP performs fine-grained classification of reasoning chains, identifying whether a chain is fully valid, partially flawed, or logically inconsistent. It jointly considers answer correctness, step-level validity, and proof-path soundness, moving beyond binary accuracy to provide deeper insights into reasoning quality. Through this design, MATP provides a more interpretable and reliable assessment of LLM reasoning performance in complex deductive tasks.

Results and Findings. We evaluate MATP through extensive experiments on two datasets: (1) a benchmark comprising reasoning tasks from PRONTOQA-OD [40], PROOFWRITER [44], and FOLIO [15], along with LLM-generated responses from 10 mainstream LLMs (five general and five reasoning models), totaling 10,830 instances; and (2) a classification dataset based on PRONTOQA-OD,

with controlled perturbations applied to standard reasoning chains. Across the benchmark dataset, MATP converts natural language sentences to FOL representations with over 99% accuracy on syntactically simpler tasks from PRONTOQA-OD and PROOFWRITER, while maintaining relatively strong performance on more linguistically complex tasks from FOLIO. In verifying individual reasoning steps, MATP outperforms prompting-based baselines by more than 42 percentage points in macro F1-score. On the classification dataset, MATP achieves 99% classification accuracy, outperforming all baselines and demonstrating its ability to distinguish fine-grained reasoning types, though occasional misclassifications remain.

Based on MATP’s evaluation and classification of all LLM-generated responses in the benchmark dataset, we observe that reasoning models, especially DeepSeek-R1, produce more logically coherent reasoning chains than general models, despite similar answer accuracy. Moreover, most evaluation failures originate from errors in the NL2FOL translation stage. We further identify and summarize common failure patterns, providing insights to guide future improvements in LLM-generated reasoning evaluation.

Contributions. The main contributions of this paper are:

- **A post-hoc evaluation framework for LLM-generated reasoning.** We propose MATP, a framework that integrates formal methods to evaluate LLM-generated reasoning by verifying both step-level correctness and overall logical consistency. MATP is open-sourced at the repository [30].
- **A novel fine-grained reasoning chain classification.** We introduce a new classification scheme that distinguishes between sound, partially flawed, and invalid reasoning chains, enabling more nuanced evaluation of LLM-generated reasoning quality.
- **Analysis of LLM-generated reasoning behaviors.** By large-scale evaluation and classification, we uncover model-specific reasoning patterns and failure modes, and identify common NL2FOL translation errors that inform future improvements.

2 Background and Motivation

2.1 Logic Reasoning in LLMs

Logical reasoning tasks performed by LLMs are generally categorized into *inductive*, *abductive*, and *deductive* reasoning. Deductive reasoning, which emphasizes rigorous inference from explicitly stated premises, is crucial in high-assurance domains like formal verification, legal reasoning, and security-critical decision-making, and forms the core focus of our work. Recent neuro-symbolic efforts have shown the feasibility of transforming natural language into formal representations and LLM-Prover integration for formal reasoning [33, 35, 57]. Inspired by these efforts, we propose a lightweight divide-and-conquer framework for assessing logical validity in LLM-generated deductive reasoning chains.

A motivating example. Figure 1 illustrates the need for systematic verification of LLM reasoning. Given a deductive reasoning task from PROOFWRITER, both GPT-3.5-Turbo and GPT-4o reach the correct conclusion but via vastly different reasoning processes. Notably, both models express their reasoning in natural language without an explicit logical structure, making it difficult to verify individual steps and their interconnections (**Challenge #1**). GPT-3.5-Turbo’s reasoning contains critical flaws: steps 3 and 7 introduce unsupported information while step 13 contradicts the premises. These

errors are masked by fluent language, illustrating **Challenge #2** in identifying subtle but consequential reasoning errors. Moreover, even when individual steps seem correct, the entire reasoning chain generated by GPT-3.5-Turbo still fails to support the conclusion, indicating that the answer might be coincidental.

2.2 Formal Foundations

We outline the technical foundations for formalizing and verifying logical reasoning using FOL and ATPs.

FOL serves as a formal language for representing and reasoning about logical statements. It extends propositional logic with predicates and quantified variables, enabling the expression of general statements about objects in a domain. The core components of FOL include: *constants* representing specific objects, *predicates* representing properties or relations, *logical connectives* such as negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$), and *quantifiers* ($\forall$ for “for all”, $\exists$ for “there exists”).

The central criterion for evaluating such reasoning is *logical validity*. A conclusion C is a logical consequence of a set of premises $P = \{P_1, P_2, \dots, P_n\}$, denoted as $P \models C$, if and only if C is true in every model (i.e., every possible interpretation of the symbols) that satisfies all premises in P . This property of validity ensures that there can be no counterexample where the premises hold true but the conclusion does not, providing a rigorous standard for correctness. In sound and complete logical systems like FOL, this semantic entailment ($\models$) is equivalent to syntactic provability ($\vdash$), meaning a valid conclusion can also be derived from the premises through a finite sequence of formal inference steps. Our work aims to automatically check if the content generated by LLMs satisfy this standard of validity with respect to the given premises.

ATPs are specialized tools designed to automatically determine the validity of logical entailments in formal systems like FOL. State-of-the-art ATPs, including Vampire [19] used in our work, primarily operate on the principle of proof by refutation. To verify the logical validity $P \models C$, the prover mechanistically attempts to find a syntactic proof for the unsatisfiability of $P \cup \{\neg C\}$, i.e., it searches for a derivation $P \cup \{\neg C\} \vdash \perp$, where $\perp$ represents a contradiction. Due to the completeness of FOL, finding such a proof provides a definitive confirmation of the original semantic entailment $P \models C$. For interoperability, our framework translates FOL formulas into the standardized TPTP (Thousands of Problems for Theorem Provers) format [43]. Using a high-performance ATP like Vampire enables the rigorous and automated validation of reasoning steps at scale.

3 Methodology

We develop a fine-grained evaluation framework for LLM-generated reasoning in deductive tasks, focusing on more than just the final answer’s correctness (§ 3.1). Specifically, MATP uses an LLM to convert both reasoning tasks and model-generated responses into formal representations (§ 3.2), employs an ATP tool and designed algorithms to verify step-level correctness and overall validity of the reasoning chain (§ 3.3), and classifies the reasoning chain into fine-grained categories (§ 3.4). An overview is shown in Figure 2.

3.1 Task Formalization

Given a set of premises and a candidate conclusion, the target LLM is prompted to determine whether the conclusion logically follows

from the premises and to generate reasoning to support its judgment. MATP takes the problem statement and the generated reasoning content as input, and performs formal verification to assess both the correctness of individual reasoning steps and the overall sufficiency of the reasoning chain. Formally, the input to MATP is defined as $D = \{P, C, L, S, A\}$, consisting of five components:

- (1) **Premises (P):** A set of factual statements $P = \{P_1, P_2, \dots, P_n\}$, where each P_i is assumed to be true for subsequent reasoning.
- (2) **Candidate Conclusion (C):** A statement that is subject to validation based on the given premises P . By design, it is guaranteed to either logically follow from P or contradict it.
- (3) **Ground Truth Label (L):** A manually annotated label $L \in \{True, False\}$ representing the correctness of the candidate conclusion C under the given premises P . Specifically, *True* indicates that $P \vdash C$, i.e., C logically follows from P , and *False* indicates that $P \vdash \neg C$, i.e., C contradicts P .
- (4) **Reasoning Steps (S):** A sequence of intermediate statements $S = (S_1, S_2, \dots, S_m)$ generated by the target LLM during its reasoning process, where each S_i represents a statement produced as part of the model’s reasoning trajectory toward validating or refuting the conclusion.
- (5) **Predicted Answer (A):** The final judgment $A \in \{True, False\}$ produced by the target LLM regarding the correctness of the candidate conclusion C , representing the model’s decision based on the reasoning steps S .

Based on the input D , MATP translates the premises P , candidate conclusion C , and reasoning steps S into FOL formulas and performs logical verification using an automated theorem prover. It produces three key outputs to facilitate comprehensive evaluation:

- (1) **Reasoning Step Correctness Annotation:** For each reasoning step, MATP assigns a correctness label $StepCorrectness(S_i) \in \{True, False, Unknown\}$, indicating whether S_i is provable, refutable, or indeterminate based on the premises P .
- (2) **Valid Proof Path Existence:** MATP searches for a subset of correct reasoning steps $S_{inProofPath} \subseteq S$ that together entail the conclusion C or its negation $\neg C$. If such a valid proof path exists, it indicates that the reasoning chain is logically sound.
- (3) **Fine-Grained Classification of the Reasoning Chain:** By analyzing the correctness of the final predicted answer, the correctness of individual reasoning steps, and the existence of a valid proof path, MATP categorizes the reasoning output into distinct fine-grained classes.

Through this principled workflow, MATP enables a rigorous and interpretable assessment of LLM reasoning capabilities, going beyond surface-level correctness to reveal potential logical errors and providing insights into the underlying reasoning structure.

3.2 NL2FOL

This module converts the given premises P , candidate conclusion C , and reasoning steps S into FOL representations to enable verification with automated theorem proving.

3.2.1 Preprocessing. To ensure the accuracy and reliability of translating NL to FOL, we first apply a series of preprocessing measures. **Interacting with LLMs.** To generate reasoning chains for subsequent formalization and verification, we employ a structured

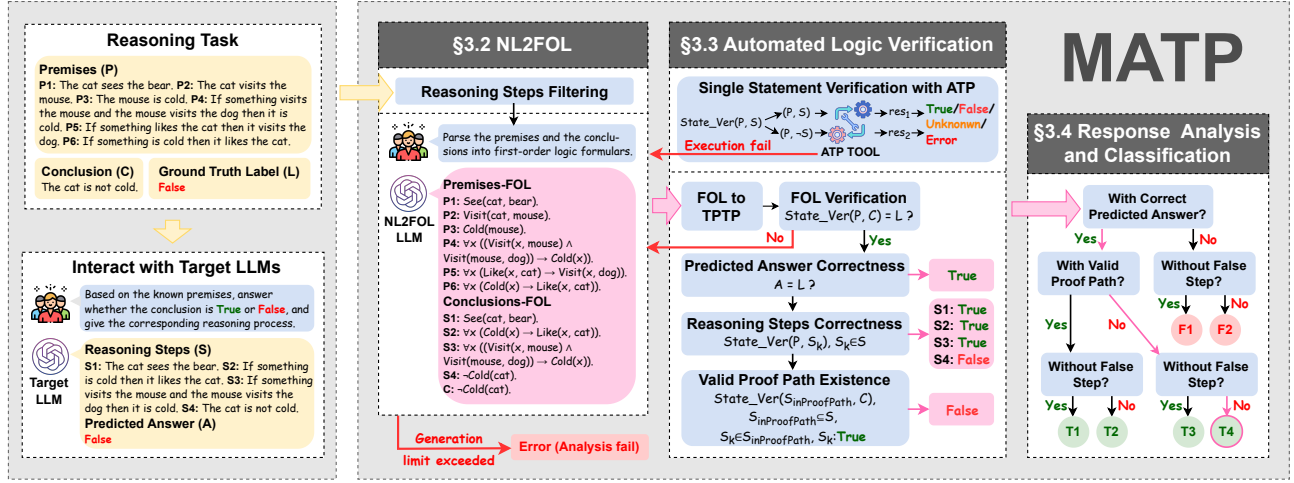


Figure 2: The workflow of MATP. Yellow nodes indicate the input, while pink nodes represent intermediate outputs.

Prompt Template for Translating NL to FOL

Parse the given premises and conclusions into first-order logic formulas.

Rules

- Each sentence must be converted into a single FOL formula.
- Use the following FOL syntax: logical conjunction of expr1 and expr2 : $\text{expr1} \wedge \text{expr2} \dots$
- Every formula must explicitly include quantifiers ($\forall x$ or $\exists x$) where applicable ...
- When an existential entity reappears with new properties, all properties (both old and new) should be conjoined within the scope of a single existential quantifier into the later formula ... (other more specific rules)

Output Format
Premises / Conclusions: fol-formula ::: (index) reference. ...

Examples

Premises: (1) The cow is cold. (2) The cow is not round. ...
Conclusions: (1) The mouse needs the cow. ... (9) The mouse eats the mouse.

Your Task
Return first-order logic formulas with NO other texts.
Premises: {premises}
Conclusions: {conclusions}

Figure 3: Prompt Template for Translating NL to FOL.

interaction with LLMs. Given premises P and a candidate conclusion C , the model is prompted to assess whether C logically follows from P through an explicitly generated reasoning process. We use a few-shot prompting strategy, providing annotated examples to guide the model’s behavior. The LLM is instructed to first produce a step-by-step reasoning process based solely on P , followed by a **True/False** decision on whether C is entailed. This answer-after-reasoning format decouples the reasoning trace from the final verdict, enabling fine-grained evaluation of both components. To ensure faithful reasoning, the prompt explicitly constrains the output to avoid unnecessary complexity, ambiguous language, or reliance on external knowledge. The full prompt design is included in [30].

Further filtering. After generating the reasoning process, the output is split into individual sentences. Subsequently, uncertain or speculative statements (e.g., “Wren is not necessarily a gopus”) are filtered out. Inspired by prior work [8], which segments reasoning content using reflection-related keywords to identify reflective phases, we extend this approach. We remove uncertain or speculative statements by detecting ambiguity-indicative expressions (e.g., “possible”, “contradict”, or “not necessarily”), without introducing additional filtering models. The remaining reasoning steps are

concise and explicit, aiding downstream processing. In particular, repetitive statements are deduplicated after converting sentences into FOL form, as direct comparison in natural language is difficult.

3.2.2 Translating Natural Language into First-Order Logic. Inspired by Logic-LM [35], which successfully leverages prompt engineering with GPT-4 for formalizing long-text reasoning tasks, we follow its prompt design framework to implement NL2FOL. While retaining its core structure, we introduce several modifications tailored to the specific requirements of our task.

Prompt Structure. As shown in Figure 3, the prompt structure comprises five components: *task definition*, *translation rules*, *output format*, *example demonstrations*, and *input instance*, detailed as follows. The “conclusions” in the prompt include both the reasoning steps S and the candidate conclusion C from the actual input.

Task definition establishes the goal of converting premises and conclusions into corresponding FOL representations. *Translation rules* define the constraints that govern the NL2FOL process, ensuring statements are translated according to FOL syntax with quality assurance strategies. These rules are flexible and can be adjusted for specific scenarios. *Output format* requires each generated output to include a formal FOL expression, an index, and the corresponding reference sentence from the input. To ensure completeness and avoid omissions, both premises and conclusions are numerically indexed, accommodating potentially lengthy inputs. *Example demonstrations* clarify the input-output relationships, reinforcing the correct application of conversion principles. *Input instance* provides the reasoning input, where “Premises” corresponds to premises P , and “Conclusions” includes reasoning steps S and the candidate conclusion C . The model is instructed to output only the required FOL formulas, without any extraneous content.

Quality Assurance Strategies. In the *Rules* component, we implement strategies to minimize common errors and improve translation quality. For **quantifier usage**, variables must have explicit existential or universal quantifiers. Parentheses are applied to complex statements to define quantifier scopes and avoid ambiguity. For existentially quantified entities, multiple statements can describe different attributes of the same entity, such as introducing property

A for entity *Tom* and later adding property B . The later translation should integrate both properties to preserve semantic completeness. To ensure **cross-step coherence in existential instantiation**, we enforce rule constraints to consistently reference entities and their predicate attributes from earlier steps, avoiding logical inconsistencies. As we translate premises, reasoning steps, and candidate conclusions in a single pass, LLMs’ comprehension aids predicate consistency. To further ensure **consistent predicate usage**, we encourage reusing existing predicates before introducing new ones, reducing redundancy and preventing predicate proliferation. To address **LLM’s confusion** between “exclusive or” and “inclusive or”, we include targeted conversion examples to clarify their distinction. Detailed implementations are provided in the full prompt [30].

3.2.3 Regeneration. The translation from natural language to FOL by LLMs is not always correct initially. If the generated FOL results in execution errors during verification, or if the verification outcome of the candidate conclusion contradicts the ground-truth label, the NL2FOL process is retried up to a fixed maximum number of attempts. If all retries fail, an error is returned, indicating failure to analyze the input. No feedback is provided during regeneration. Since pinpointing semantic errors in translation is challenging, we cannot provide useful and specific feedback. While using the entire output from the previous generation as feedback is another option, experiments in § 4.5 show its minimal impact.

3.3 Logical Verification

This module aims to perform logical verification on FOL representations of input statements (P, C, S) using the automated theorem prover. This process assigns correctness labels to each reasoning step and determines whether a valid proof path exists.

FOL to TPTP. We convert FOL formulas into TPTP format for compatibility with standard ATP tools. This is done via a Python script that enforces a deterministic one-to-one mapping between FOL and TPTP representations. Preprocessing steps are applied to align the FOL formulas with TPTP syntax, including replacing Unicode symbols with ASCII characters, normalizing predicate and constant names, and handling unsupported logical connectives (e.g., exclusive disjunction $A \oplus B$ is rewritten as $(\neg A \wedge B) \vee (A \wedge \neg B)$). The script also performs lightweight syntax repair—such as correcting missing parentheses or misused connectives—to improve robustness and reduce parsing errors in downstream verification.

Each formula is translated into TPTP’s FOF format: “fof(*name*, *role*, *formula*)”, where *name* is a unique identifier for the logical statement, *role* specifies whether it is an axiom or a conjecture, and *formula* is the FOL expression rewritten in TPTP syntax. All premises are assigned as axioms, and the candidate conclusion as a conjecture. Unique identifiers follow the pattern “*premise_index*” and “*conclusion*”. We first translate natural language into FOL before TPTP because TPTP syntax is more complex and verbose, making direct NL2TPTP translation more error-prone. Converting FOL to TPTP helps correct syntax errors and ensures compatibility with theorem provers, improving verification reliability.

Provability Verification. A core component of the verification process is to determine whether a statement S is provable from a set of premises P . There are three possible outcomes for S : *provable*, which means S is logically entailed by P and it holds true whenever

P is true; *refutable*, which means S contradicts P and is always false when P is true; *indeterminate*, which means S is neither provable nor refutable based on P . To determine this, we invoke the prover to simultaneously check whether $P \vdash S$ or $P \vdash \neg S$. If either holds, S is classified as *True* or *False* accordingly. If neither holds (i.e., both $P \cup S$ and $P \cup \neg S$ are consistent), S is labeled *Unknown*. **Algorithm 1** summarizes this procedure, with the outputs *True*, *False*, and *Unknown* corresponding to the three provability outcomes. In particular, if both S and $\neg S$ are provable, it indicates a contradiction within P , and we return an *Error*.

Algorithm 1 Single Statement Verification using ATP

Require: Premises $P = \{P_1, P_2, \dots, P_n\}$, candidate statement S
Ensure: Check Result (*True*, *False*, *Unknown*, or *Error*)

```

1: function VERIFYSINGLESTATEMENT( $P, S$ )
2:    $res_1 \leftarrow \text{ATPRUN}(P, S)$  and  $res_2 \leftarrow \text{ATPRUN}(P, \neg S)$ 
3:   if Refutation in  $res_1$  and Refutation in  $res_2$  then
4:     return Error
5:   if Refutation in  $res_1$  then
6:     return True
7:   if Refutation in  $res_2$  then
8:     return False
9:   return Unknown

```

► Contradiction exists in P
 ► S is verified
 ► $\neg S$ is verified

Verification of the Reasoning Steps. We begin by applying VERIFYSINGLESTATEMENT to the premises P and the candidate conclusion C to check alignment with the ground-truth label L . A mismatch may arise from: (1) a semantically incorrect but syntactically valid NL2FOL translation that causes the ATP to produce a judgment on C inconsistent with L , or (2) an erroneous ground-truth label annotation. Since these cases cannot be automatically differentiated or corrected, the process reverts to the NL2FOL stage for regeneration. After verifying the conclusion, we assess the correctness of each intermediate reasoning step S_i under the same premises P . If a step is refuted or indeterminate, it is recorded as a potential logical error. **Algorithm 2** outlines this process. The algorithm returns both the correctness of the predicted answer and the correctness annotation for each reasoning step. Notably, some incorrect steps may not affect the final judgment, while others may compromise the entire reasoning chain. To distinguish these cases, further analysis is conducted to determine whether a valid proof path exists within the reasoning steps.

Algorithm 2 Reasoning Steps Verification

Require: Premises $P = \{P_1, P_2, \dots, P_n\}$, candidate conclusion C , ground truth label L , reasoning steps $S = \{S_1, S_2, \dots, S_m\}$, predicted answer A
Ensure: Returns *AnswerCorrectness*, step correctness annotation *Scorrectness*

```

1: function VERIFYREASONINGSTEPS( $P, C, L, S, A$ )
2:   if VERIFYSINGLESTATEMENT( $P, C$ )  $\neq L$  then
3:     return Error
4:    $Scorrectness \leftarrow []$ 
5:   for  $i \leftarrow 1$  to  $m$  do
6:     APPEND( $Scorrectness$ , VERIFYSINGLESTATEMENT( $P, S_i$ ))
7:    $AnswerCorrectness \leftarrow (L = A)$ 
8:   return  $AnswerCorrectness, Scorrectness$ 

```

► NL2FOL translation error or labeling error
 ► Initialize step correctness list
 ► Check if the answer matches

Verification of the Reasoning Proof Path. Beyond evaluating individual reasoning steps, we also assess whether the reasoning process contains all the necessary steps to logically support the conclusion label. In other words, we determine whether the reasoning chain includes a valid proof path leading to the correct classification of the conclusion. To do this, we iteratively construct a candidate proof path by including only those *True*-labeled steps that introduce new facts not derivable from previously included steps. Formally, the proof path is a subset $S_{inProofPath} \subseteq S$, where a

step S_i is added if and only if it is labeled *True* and not entailed by $S_{inProofPath}$. After processing all steps, we check if the truth value of the candidate conclusion C matches the ground truth label L , when evaluated under the proof path $S_{inProofPath}$. If *True*, it indicates that reasoning process contains a valid proof path that supports the conclusion label. Otherwise, the proof path is considered invalid. This verification process is detailed in [Algorithm 3](#).

Algorithm 3 Reasoning Proof Path Verification

Require: Candidate conclusion C , ground truth label L , reasoning steps $S = \{S_1, S_2, \dots, S_m\}$, step correctness annotation $S_{correctness}$
Ensure: Returns *HasValidProofPath*

```

1: function VERIFYREASONINGPROOFPATH( $C, L, S, S_{correctness}$ )
2:    $S_{inProofPath} \leftarrow []$ 
3:   for  $i \leftarrow 1$  to  $m$  do
4:     if  $S_{correctness}_i = \text{True}$  then
5:       if  $\text{VERIFYSINGLESTATEMENT}(S_{inProofPath}, S_i) = \text{Unknown}$  then
6:         APPEND( $S_{inProofPath}, S_i$ )           ▶ Construct the proof path
7:    $\text{HasValidProofPath} \leftarrow (\text{VERIFYSINGLESTATEMENT}(S_{inProofPath}, C) = L)$ 
8:   return HasValidProofPath

```

If the automated theorem prover encounters an execution error indicating syntactic flaws in the generated FOL content, we return to the NL2FOL stage for regeneration.

3.4 Response Analysis and Classification

After the first two stages, inputs exceeding the maximum NL2FOL retries are categorized as **Error**, indicating that our framework cannot handle them. For successfully processed inputs, this module performs a fine-grained classification of reasoning chains based on logical verification to evaluate the reasoning process comprehensively. Instead of only assessing the final predicted answer, we analyze both individual reasoning steps and the overall reasoning chain to identify potential logical flaws. This allows us to distinguish between responses that are correct by chance and those based on valid reasoning. After logical verification, we obtain three key evaluation dimensions: (1) correctness of the predicted answer, (2) correctness of each reasoning step, and (3) existence of a valid proof path within the reasoning chain. Based on these dimensions, we define six reasoning categories, as illustrated in [Figure 2](#).

For responses with incorrect predicted answer, we classify them into two categories based on whether intermediate reasoning steps contain errors. Any reasoning step labeled as *False* or *Unknown* is considered erroneous. **F1** represents cases where all reasoning steps are correct but the final answer is incorrect, suggesting that the target model likely fails to construct a valid proof path. **F2** covers cases where both the final answer and at least one reasoning step are incorrect, suggesting that intermediate errors may mislead the reasoning process or that the model fails to form a coherent proof.

For responses with correct predicted answers, we assess both the correctness of individual steps and the validity of the reasoning chain. **T1** denotes the ideal cases where all steps are correct and a valid proof path exists. **T2** includes cases with some erroneous steps, yet the proof path remains valid—indicating partial robustness. **T3** represents cases where all reasoning steps are correct, but the reasoning chain does not constitute a valid proof, suggesting that the model correctly infers the answer but fails to provide a logically complete reasoning path. **T4** includes responses with step-level errors and no valid proof path, raising concerns that the correct answer may be reached by chance or flawed reasoning.

4 Evaluation

Our evaluation targets the following research questions:

- **RQ1 (Effectiveness):** Can MATP accurately verify the correctness of reasoning steps generated by LLMs?
- **RQ2 (Reasoning Chain Classification):** Can MATP effectively classify different types of reasoning chains?
- **RQ3 (Model Reasoning Analysis):** How do different LLMs perform in terms of reasoning quality, and what are their characteristic behaviors when generating reasoning chains?

4.1 Experimental Setup

Benchmark Dataset. We construct our benchmark from three reasoning-focused datasets, each designed for natural language inference tasks. PRONTOQA-ODD [40] and PROOFWRITER [44] focus on logical reasoning within structured ontologies, emphasizing categories, attributes, and rule-based inference, while FOLIO [15] centers on realistic semantic reasoning with knowledge about people, locations, quantities, and events. Each test case in these datasets consists of premises, a candidate conclusion, and a ground-truth label (*True*, *False*, or *Unknown*). Test cases in PRONTOQA-ODD and PROOFWRITER use structured natural language, while those in FOLIO use expressions closer to everyday human language.

- **PRONTOQA-ODD** [40] (384 test cases selected) is a synthetic dataset where all conclusions are labeled *True*. Tasks are categorized by the minimal number of reasoning steps required; we sample from the 4-hop subset to ensure higher complexity.
- **PROOFWRITER** [44] (344 test cases selected) uses more natural language and complex logic. We sample from its 5-hop test set, excluding tasks with *Unknown* label due to unclear entailment.
- **FOLIO** [15] (355 test cases selected) is a human-authored dataset grounded in real-world facts. We sample from the training set, retaining only *True/False* cases and discarding instances where the conclusion trivially repeats a premise.

To balance dataset scale and focus on higher-hop instances that elicit richer model responses, we randomly sample test cases from the maximum-hop subset of PRONTOQA-ODD and PROOFWRITER, rather than using all available data. For FOLIO, which lacks hop-based partitioning, we randomly sample from its training set. For each selected test case, we generate reasoning chains and predicted answers using 10 representative LLMs (detailed in the next section). The prompting template used for generation is provided in [30]. This results in a total of 10,830 benchmark instances. Each instance includes the premises, candidate conclusion and ground-truth label provided by the test case, as well as the reasoning chain and predicted answer generated by the target LLM. This benchmark is used to evaluate the effectiveness of MATP (RQ1) and to analyze reasoning chain categories across models (RQ3).

Models Under Test. To ensure a comprehensive evaluation, we test 10 LLMs with MATP, including five general models and five reasoning models. The general models include GPT-3.5-turbo, GPT-4o [1, 18], LLaMA3.1-8B-chat-hf, LLaMA3.1-70B-chat-hf [13] and Qwen2.5-7B-Instruct [47]. The reasoning models include QwQ-32B [38], DeepSeek-R1-Distill-Qwen-7B, DeepSeek-R1-Distill-Qwen-32B¹, DeepSeek-R1 [14] and GPT-o4-mini [34].

¹We use DS-R1-Qwen7B/32B to represent DeepSeek-R1-Distill-Qwen-7B/32B for short.

Table 1: Evaluation of MATP on the Benchmark: NL2FOL Translation Accuracy and Reasoning Step Correctness Classification Performance. NL2FOL accuracy (§ 4.2.1) is evaluated using four metrics—ER (Execution Rate), EA (Execution Accuracy), FB (FOL BLEU), and LE (Logical Equivalence)—while reasoning step correctness (§ 4.2.2) is measured by Macro F1 score. All values are reported as percentages (%), with higher scores indicating better performance. In the metric rows, GPT-4o and DS-R1 refer to prompt-based baseline methods implemented using GPT-4o and DeepSeek-R1, respectively.

Dataset	PrOntoQA-OOD (384)							PROOFWRITER (344)							FOLIO (355)						
					Macro F1 Score							Macro F1 Score							Macro F1 Score		
	ER	EA	FB	LE	MATP	GPT-4o	DS-R1	ER	EA	FB	LE	MATP	GPT-4o	DS-R1	ER	EA	FB	LE	MATP	GPT-4o	DS-R1
GPT-3.5	99.22	100.0	100.0	100.0	87.13	54.33	39.94	99.71	99.71	83.88	99.91	100.0	43.03	38.39	96.34	69.59	36.98	88.37	91.93	57.11	43.21
GPT-4o	98.96	100.0	100.0	100.0	100.0	49.75	47.92	99.71	99.42	84.14	100.0	100.0	33.08	33.33	96.06	70.09	37.11	87.78	100.0	42.23	31.06
LLaMA3.1-8B	99.22	100.0	100.0	100.0	100.0	38.53	49.74	99.71	100.0	83.88	99.91	100.0	46.77	40.00	95.49	70.50	36.84	88.52	89.87	68.01	47.08
LLaMA3.1-70B	99.22	99.74	100.0	100.0	55.42	49.59	69.69	99.71	97.38	83.69	99.91	98.61	46.10	43.05	94.37	71.64	36.46	88.20	93.51	38.53	42.12
Qwen2.5-7B	98.96	100.0	100.0	100.0	100.0	44.43	32.80	99.42	99.42	84.14	100.0	96.34	52.80	47.29	97.18	72.46	36.35	88.15	79.69	49.34	38.40
QwQ-32B	99.22	100.0	100.0	100.0	100.0	49.37	49.79	99.13	99.41	84.28	100.0	100.0	72.70	49.27	95.49	71.09	36.63	88.16	88.75	37.29	36.55
DS-R1-Qwen7B	99.22	99.74	100.0	100.0	100.0	44.27	39.96	99.42	98.25	84.14	100.0	98.84	62.44	36.66	96.34	70.76	37.81	88.65	72.08	41.72	39.66
DS-R1-Qwen32B	99.22	99.74	100.0	100.0	100.0	49.58	83.13	99.42	99.42	84.28	100.0	66.47	32.52	32.42	96.62	71.43	37.29	88.92	72.82	38.53	33.70
DeepSeek-R1	99.22	99.74	100.0	100.0	100.0	49.82	100.0	99.42	99.42	84.21	100.0	52.16	49.19	49.35	97.18	70.72	37.06	87.82	61.33	32.82	31.33
GPT-o4-mini	98.18	100.0	100.0	100.0	100.0	48.19	100.0	99.71	100.0	84.07	100.0	100.0	54.96	54.49	95.49	70.50	36.58	88.45	94.24	46.13	32.40
Average	99.06	99.90	100.0	100.0	94.26	47.79	61.30	99.53	99.24	84.07	99.97	91.24	49.36	42.43	96.06	70.88	36.91	88.30	84.42	45.17	37.55

Model Configuration. We set *temperature* to 0 for stable and consistent reasoning outputs, with *top-p* = 1 and *top-k* = 0 to favor high-probability tokens. An exception is GPT-o4-mini, which uses a fixed temperature of 1 due to model limitations.

Baseline. We compare our method against a prompting-based baseline using GPT-4o and DeepSeek-R1 (RQ1 and RQ2), following [59]. The prompt elicits two outputs: (1) step-by-step judgments on reasoning steps (*True*, *False*, or *Unknown*); and (2) a final decision on whether the reasoning supports the predicted answer (*Yes* or *No*), both with explanations. The full prompt is provided in [30].

Implementation. Based on MetaGPT [16], a development framework for multi-agent systems that coordinates tool calls and manages pipeline stages, we integrate existing components to implement the evaluation framework as an automated toolchain. Specifically, we use GPT-4o for the NL2FOL translation, leveraging its strong performance on long and complex natural language tasks, and limit generation attempts to three per input. For automated verification, we adopt the widely used automated theorem prover VAMPIRE [19]. All implementation details are available at [30].

Running Environment. All experiments run on a server with Ubuntu 22.04, equipped with two 64-core AMD EPYC 7713 CPUs, 512 GB RAM, and two NVIDIA A100 PCIe 80GB GPUs.

4.2 RQ1: Effectiveness

We conduct evaluations on the benchmark dataset to assess the effectiveness of MATP. The evaluation focuses on two aspects: the accuracy of NL2FOL translation and the ability to identify the correctness of individual reasoning steps.

4.2.1 Accuracy of NL2FOL translation. We evaluate NL2FOL translation at both the overall and sentence levels.

Overall Level Metrics. Following Logic-LM [35], we use Execution Rate (ER) and Execution Accuracy (EA) to evaluate NL2FOL translation across the entire benchmark. ER measures the proportion of FOL outputs that can be successfully executed by VAMPIRE, reflecting syntactic correctness. EA measures how often VAMPIRE’s verdict matches the ground truth label when execution succeeds, indicating semantic preservation.

Sentence Level Metrics. We adopt FOL BLEU (FB) and Logical Equivalence (LE) from LogicLLaMA [56] to assess sentence-level

conversion. FB applies FOL-specific tokenization to compute similarity scores, while LE determines whether the generated and reference FOL formulas are logically equivalent. For datasets with reference FOLs (FOLIO), we compute FB and LE across all premises. For others (PrOntoQA-OOD, PROOFWRITER), we manually annotate reference FOLs for 300 premises per dataset for metric evaluation, with cross-verification by three authors to ensure consistency.

Results. As shown in Table 1, both ER and EA on PrOntoQA-OOD and PROOFWRITER are close to 100%, indicating strong syntactic and semantic alignment. In contrast, FOLIO shows high ER (96.06%) but lower EA (70.88%), suggesting challenges in preserving semantics for complex statements. At the sentence level, FB and LE scores on PrOntoQA-OOD and LE on PROOFWRITER are near 100%. However, PROOFWRITER’s FB drops to 84.07%, mainly due to mismatches in forms (e.g., “Like” vs. “Likes”) between model outputs and base-form annotations, highlighting FB’s sensitivity to surface variations. On FOLIO, the FB score drops to 36.91%, while LE remains relatively high (88.30%), reflecting logical equivalence despite low textual similarity. This discrepancy is understandable, as for complex sentences, the choice of predicate or constant names is unlikely to consistently match the expected form.

To better understand the low EA on FOLIO, we conducted a manual review of cases where the MATP’s labels for candidate conclusion differ from the ground truth labels. The key factors contributing to this issue will be analyzed in detail in § 4.4.3.

4.2.2 Ability to identify the correctness of individual reasoning steps. This evaluation focuses on whether each reasoning step is correctly labeled by MATP, and how it performs versus the baseline.

Metric. We formulate the task as a three-class classification problem (*True*, *False*, *Unknown*) and evaluate it using macro F1-score. Due to annotation cost, we sample 20 ER-contributing tasks per dataset for each of ten models (600 samples total). Each reasoning step correctness is annotated by three authors, and the aggregated labels are used to compute the final macro F1 score.

Results. The detailed results are shown in Table 1. MATP consistently outperforms both GPT-4o and DeepSeek-R1 baselines in classifying reasoning step correctness. On PrOntoQA-OOD and PROOFWRITER, MATP achieves 94.26% and 91.24% F1, significantly higher than GPT-4o (47.79%, 49.36%) and DeepSeek-R1 (61.30%,

42.43%). Even on the more informal FOLIO dataset, MATP maintains 84.42% F1, while the baselines drop below 46%. We further analyze the reasons behind the poor performance of the baselines. Specifically, the baselines often mislabel correct steps as *False* or *Unknown* when the inference is implicit or requires multi-hop reasoning. They also struggle to distinguish between *False* and *Unknown*, resulting in frequent confusion between the two.

ANSWER to RQ1

MATP verifies reasoning steps by accurately translating NL to FOL and identifying correctness, outperforming prompting-based baselines across all datasets. It performs well on PRONTOQA-OOD and PROOFWRITER, but struggles on FOLIO due to more complex language patterns.

4.3 RQ2: Reasoning Chain Classification

To evaluate whether MATP can accurately classify reasoning chains, we conduct evaluations on a classification dataset and compare its performance with baselines.

Classification Dataset. We construct a classification dataset from 200 PRONTOQA-OOD tasks. Starting from the standard reasoning chains provided by PRONTOQA-OOD (T1-type), we apply targeted mutations to obtain different types of reasoning chains: to generate T2-type chains, we insert a negated premise to introduce false reasoning step; for T3-type chains, we remove key steps and inject irrelevant content to break the logical flow; for T4-type chains, we apply both T2-type and T3-type mutations. Each variant is created for 50 tasks to ensure balanced coverage across the four types.

Results. Figure 4 presents the classification accuracy of MATP versus the baselines. As shown, MATP achieves 100% accuracy on T1–T3 and 96% on T4. A review of the two T4 misclassifications attributes the errors to minor NL2FOL translation deviations, suggesting that with correct translation, MATP can consistently classify reasoning chains accurately. In contrast, the GPT-4o-based baseline achieves lower accuracy—78%, 72%, 84%, and 90% for T1–T4, respectively. Error analysis reveals two main issues: (1) 54 correct steps were misjudged as incorrect, causing 14 T1/T3 cases to be classified as T2/T4; and (2) it struggles to assess whether steps sufficiently support the conclusion, leading to 20 T1/T2 cases misclassified as T3/T4 and 11 T3/T4 cases misclassified as T1/T2. The DeepSeek-R1-based baseline performs better, achieving 96%, 90%, 92%, and 82% accuracy. Most misclassifications arise from failure to assess whether steps sufficiently support conclusions—6 T1/T2 cases were mistaken for T3/T4, and 13 T3/T4 as T1/T2—mainly due to its weaker instruction-following ability, often defaulting to global premise-to-conclusion matching.

While 100% classification accuracy can be guaranteed with perfect translation, we cannot ensure flawless NL2FOL translation in practice. Consequently, MATP cannot ensure absolute correctness in real-world applications. Nevertheless, it achieves significantly higher accuracy than baselines. We also record the average processing time per task for all tools. GPT-4o takes around 6s per task, DeepSeek-R1 is significantly slower at 96s, while MATP maintains a comparable speed of approximately 9s.

ANSWER to RQ2

MATP achieves perfect classification given accurate NL2FOL translation, clearly outperforming the prompting-based baselines.

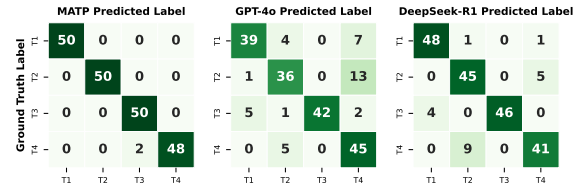


Figure 4: Heatmap of Reasoning Chain Classification Performance for MATP and Baselines.

4.4 RQ3: Model Reasoning Analysis

Using MATP’s reasoning chain classifications, we analyze LLM performance across tasks, focusing on reasoning quality and model-specific patterns. We first compare model performance across datasets to identify strengths, limitations, and reasoning tendencies, then analyze structural patterns within each category, and finally examine cases where MATP fails to identify typical causes. This analysis reveals key reasoning bottlenecks and guides future improvements in LLM reasoning.

4.4.1 Analysis of Model Performance. The overall category distribution is shown in Figure 5, where Error denotes cases that MATP fails to evaluate. The average failure rate is 1.04% on PRONTOQA-OOD, 1.22% on PROOFWRITER, and 31.92% on FOLIO. The notably higher rate on FOLIO suggests that MATP is more sensitive to complex, natural language structures, mainly due to challenges in NL2FOL translation. The specific failure reasons are detailed in § 4.4.3, and these cases are excluded from subsequent analysis.

While reasoning and general models achieve similar answer accuracy on simple datasets like PRONTOQA-OOD, their reasoning quality differs markedly. Reasoning models (e.g., DeepSeek-R1, QwQ-32B) consistently generate valid reasoning chains (T1–T2), with DeepSeek-R1 reaching over 96% T1 on PRONTOQA-OOD. In contrast, general models (e.g., LLaMA3.1–8B) often rely on shallow or inconsistent logic, lacking valid proof paths despite high accuracy. General models also show issues like redundant logic (LLaMA) or overuse of irrelevant premises (GPT-3.5), leading to correct but weakly justified answers. Even reasoning models struggle on challenging datasets like PROOFWRITER, where incomplete or invalid chains (T3/T4) remain. On FOLIO, T2 and T4 increases across all models. This stems partly from MATP’s difficulty in handling truly natural language, and partly from the dataset’s reliance on background knowledge not always captured in the explicit premises. Model scale also affects performance. Larger models (e.g., LLaMA3.1–70B) improve T1 ratios and reduce errors but still lag behind reasoning-specialized models in logical coherence.

In summary, answer accuracy alone fails to capture reasoning strength. Our classification strategy reveals critical differences in logical validity and robustness, highlighting the need to evaluate reasoning processes, not just final answers.

4.4.2 Analysis of Reasoning Chain Categories. We further analyze each category to identify common patterns and provide representative examples in Table 2.

T1 chains that are fully correct in both step-level logic and overall consistency, representing ideal outputs. Nonetheless, even when satisfying our classification criteria, there remain minor issues that warrant further discussion, such as redundant steps that are correct

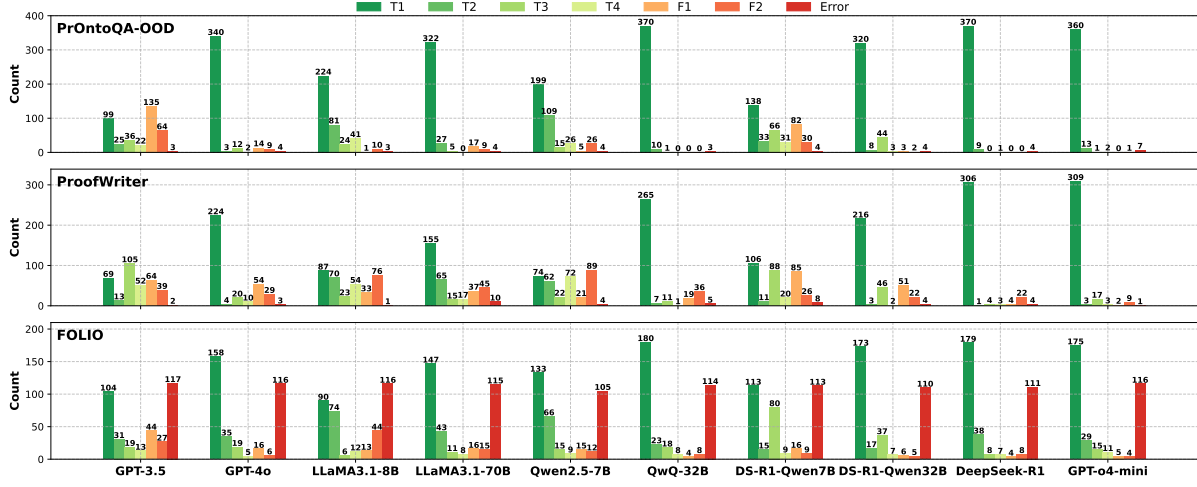


Figure 5: Category Distribution of LLM-Generated Reasoning Chains on Benchmark Datasets.

Table 2: Special Cases of Reasoning Chain Categorization with Representative Examples. In the Reasoning Response, sentences highlighted in orange are considered *Unknown* relative to the task premises, while black sentences are *True*.

Description	Reasoning Response	Task Name
T2: If the <i>Unknown</i> step is excluded, this answer is a standard response.	GPT-4o: 1) The tiger eats the dog. 2) If someone eats the dog then the dog needs the bear. 3) The dog eats the dog. 4) The dog needs the bear. 5) There is evidence to conclude that the dog needs the bear.	Task ①: PROOFWRITER- RelNeg-OWA-D5- 226-Q3
T3: The conclusion that needs to be proven is "Fiona is not big". However, the model is proving "Anne is big".	GPT-3.5: ... 5) If Anne is white then Anne is big. 6) There is evidence to conclude that Anne is big.	Task ②: PROOFWRITER- AttNeg-OWA-D5- 1003-Q5
T4: The model introduce an <i>Unknown</i> information to fabricate a valid proof path.	DS-R1-Qwen7B: 1) Sally is a dumpus. 2) Sally is a gorpup. 3) Every gorpup is a zumpus. 4) Therefore, Sally is a zumpus. 5) Every zumpus is a romrus. 6) Therefore, Sally is a romrus.	Task ③: PrOntoQA-OOD- 4hop-Composd- random-noadj-69-4
F1: The model's reasoning contains enough information to derive the conclusion, but it incorrectly claims it cannot.	QwQ-32B: 1) Fiona is quiet. 2) Quiet things are green. 3) Fiona must be green. 4) Fiona is rough. ... 9) Therefore, Fiona is not green.	Task ④: PROOFWRITER- AttNeg-OWA-D5- 630-Q4
F1: The model's reasoning steps are entirely correct, but the final answer is incorrect.	GPT-4o: 1) Bob is quiet. 2) Bob is smart. 3) All smart things are green. 4) Bob is green. 5) Green things are white. 6) Bob is white. 7) There is evidence to conclude that Bob is white.	Task ⑤: PROOFWRITER- AttNeg-OWA-D5- 327-Q5
F1: The model enters a loop during reasoning, causing the inference process to stall.	LLaMA3.1-8B: 1) Every lorpus is a numpus. 2) Every lorpus is a numpus. ... 20) Every lorpus is a numpus. ...	Task ⑥: PrOntoQA-OOD- 4hop-Composd- random-noadj-22-7

but irrelevant, and skipped steps where a step relies on multiple unstated premises. Such cases are illustrated in GPT-4o's output in Figure 1 (step 1 and step 4).

T2 responses resemble T1 but contain one or more incorrect steps that do not affect the final conclusion. Once removed, the remaining steps form an entirely correct and valid proof (i.e., task ①).

T3 chains consist of correct steps but fail to form a valid proof path. This often results from implicit reasoning, where key premises are omitted, or accidental correctness, where an unrelated conclusion is proven that happens to share the same truth value (i.e., task ②). Models often default to answering *False* when uncertain, which may also produce coincidentally correct answers.

T4 exhibits the same issues as T3 but also includes incorrect reasoning steps. These outputs are typically disorganized and incoherent. Some cases even introduce non-existent facts to fabricate plausible chains (i.e., task ③).

F1 and F2 responses both yield incorrect final predicted answers. Reasoning chains in F1 contain only correct steps, whereas in F2 include at least one incorrect step. Common issues include incomplete reasoning due to missing or poorly integrated premises (i.e., task ④). We also observe special failure modes: models may derive the correct conclusion but answer it incorrectly (i.e., task ⑤), or enter repetitive loops that stall the reasoning process (i.e., task ⑥).

Table 3: MATP Misclassifications on DeepSeek-R1's Response for folio-train-57-168. Red indicates MATP's erroneous outputs, while green marks the expected correct ones.

Task Info and Reasoning Response	Step-FOL Generated by GPT-4o	MATP Output
Conclusion: Leo is an animal. (True)	1) $\forall x(pet(x) \rightarrow animal(x))$	S1: True
Filtered Resoning Steps:	2) $pet(leo) \wedge dog(leo) \wedge naughty(leo) \wedge haspet(charlie, leo)$	S2: True
1) All pets are animals.	3) $\forall x(dog(x) \rightarrow pet(x))$	S3: Unknown
2) Charlie has a naughty pet dog named Leo.	3) $\forall x(pet(x) \wedge dog(x) \rightarrow pet(x))$	S3: True
3) A pet dog is a type of pet.	4) $\forall x((pet(x) \wedge animal(x)) \rightarrow (pet(Leo) \rightarrow animal(Leo)))$	S4: True
4) Since all pets are animals, Leo being a pet implies Leo is an animal.		Cate: T2
		Cate: T1

In summary, T1 and T2 exhibit the most reliable reasoning, while T3 and T4 are less stable. F1 shows potential for improvement, while F2 is the most flawed. Overall reasoning quality follows: T1 > T2 > T3 > T4 > F1 > F2. Manual inspection also reveals a few misclassifications. As shown in Table 3, an NL2FOL translation error in one step leads to T1 being mislabeled as T2. While the classification is not perfectly accurate, it reliably captures overall reasoning patterns and pinpoints potential error steps, offering clearer consistency and interpretability than prompt-based baselines.

4.4.3 Analysis of Error Cases. Most evaluation failures originate from NL2FOL translation errors. Table 4 presents representative cases. While translation from PrOntoQA-OOD and PROOFWRITER is largely accurate, occasional issues remain, such as incorrect quantifier scope (i.e., case ①) or misinterpreting "or" as a conjunction when linking multiple entities (i.e., case ②). In contrast, FOLIO introduces more diverse and complex errors due to its rich natural language. We summarize common error patterns as follows.

Information Omission. Key details are omitted when sentences contain multiple facts (case ③).

Table 4: Representative Error Cases in NL2FOL Translation. Key erroneous parts are highlighted in red.

Description	NL Sentences	FOL Representation	Dataset
The domain of the universal quantifier in FOL is wrong.	If someone is smart and not kind then they are young.	$\forall x(\text{smart}(x) \wedge \neg \text{kind}(x)) \rightarrow \text{young}(x)$	Case ❶ from PROOFWRITER
The wrong use of $\wedge$ instead of $\vee$ for “or”.	Alex is a gorpus, a sterpus, a numpus, or a gorpus.	$\text{Gorpus}(\text{alex}) \wedge \text{Sterpus}(\text{alex}) \wedge \text{Numpus}(\text{alex}) \wedge \text{Gorpus}(\text{alex})$	Case ❷ from PbOntoQA-OOD
<i>BoyBand(bto4u)</i> which is effective for reasoning candidate conclusion is omitted.	Show Your Love is a song recorded by the South Korean boy band BtoB 4u.	$\text{Song}(\text{showyourlove}) \wedge \text{RecordedBy}(\text{showyourlove}, \text{bto4u})$	Case ❸ from FOLIO
The predicate is used improperly, where the same word “referee observer” in the context is represented by different predicates.	Brian Winter was appointed as a referee observer after his retirement. Some football referees become referee observers .	$\text{AppointedObserver}(\text{brian}). \exists x(\text{FootballReferee}(x) \wedge \text{RefereeObserver}(x))$	Case ❹ from FOLIO
“Either...or...” represents XOR, which differs from OR and should be denoted using $\oplus$.	Everything displayed in the collection is either a plant or an animal.	$\forall x(\text{Displayed}(x) \rightarrow (\text{Plant}(x) \vee \text{Animal}(x)))$	Case ❺ from FOLIO
The use of $>$ is ungrammatical and should be written as <i>MoreRevenue(airbus, boeing)</i> .	Airbus made more revenue than Boeing last year.	$\text{Revenue}(\text{airbus}, \text{lastyear}) > \text{Revenue}(\text{boeing}, \text{lastyear})$	Case ❻ from FOLIO

Improper Predicate Usage. The same concept may be mapped to different predicates, causing semantic inconsistency (i.e., case ❹). **XOR vs. OR Confusion.** Expressions involving exclusive choice, such as “either...or...”, are often mistakenly represented as disjunction ($\vee$) rather than exclusive or ($\oplus$), even when explicitly prompted (i.e., case ❺).

Inadequacy in Handling Domain-Specific Expressions. Domain-specific expressions involving comparatives, superlatives, mathematics, or time are often poorly handled, resulting in ungrammatical FOL (i.e., case ❻).

Annotation Errors. In addition to the extraction issues above, we also observe annotation errors in the FOLIO dataset during manual inspection. Figure 6 provides an example of such a mislabeling. We find that the FOL ground truth representation of the fourth premise in the task is incorrectly given as “ $\forall x((\text{Professional}(x) \wedge \text{HighWinRatio}(x)) \rightarrow \text{Big3}(x))$ ”, which may be the cause of the labeling error. Similar issues are also found in tasks such as folio-train-63-186, 167-480, 167-479, 302-746, 470-1359, and 470-1360.

Premises	MATP Check Result
1) Rafa Nadal was born in Mallorca. 2) Rafa Nadal is a professional tennis player. 3) Nadal's win ratio is high. 4) All players in the Big 3 are professionals who have a high win ratio.	Premises-FOL: 1) $\text{BornInMallorca}(\text{rafa})$ 2) $\text{Professional}(\text{rafa}) \wedge \text{TennisPlayer}(\text{rafa})$ 3) $\text{HighWinRatio}(\text{rafa})$ 4) $\forall x(\text{Big3}(x) \rightarrow (\text{Professional}(x) \wedge \text{HighWinRatio}(x)))$ Conclusion-FOL: $\text{Big3}(\text{rafa})$ Vampire Result: Unknown
Conclusion & Ground Truth Label Nadal is in the Big 3. $\rightarrow$ True	

Figure 6: Example of Annotation Error in FOLIO. The task ID is folio-train-34-99.

ANSWER to RQ3
MATP reveals clear differences in reasoning behavior across LLMs. Despite similar answer accuracy, reasoning models yield more coherent logic than general models, highlighting the need to assess reasoning quality beyond final answers.

4.5 Ablation Study

We conduct an ablation study comparing MATP’s performance with and without feedback during NL2FOL regeneration.

Setup. As shown in Figure 5, most analysis failures occur on FOLIO. Therefore, we evaluate the responses of two models, GPT-4o and DeepSeek-R1, on 355 tasks from FOLIO. Since pinpointing specific errors is difficult, we use the entire previous generation as feedback.

We extend the original prompt in Figure 3 by adding a *Feedback* field under *Input instance*, containing the prior NL2FOL translation with conversion errors. This field helps the model avoid similar mistakes during regeneration. Similarly, the maximum number of regeneration attempts is set to three. We focus on the number of failures in both feedback and non-feedback settings.

Results. As shown in Table 5, using Feedback has minimal effect, with non-feedback settings sometimes yielding better results. Additionally, feedback increases resource consumption. Therefore, feedback is excluded during MATP’s actual implementation.

Table 5: Impact of Feedback on MATP Failures.

Model	With Feedback	Without Feedback
GPT-4o	118	116
DeepSeek-R1	119	111

5 Discussion

5.1 Future Enhancement

This paper proposes (1) a framework for translating informal LLM reasoning into formal logic and verifying with ATP tools, and (2) a multi-level classification strategy beyond binary answer correctness. Thanks to the modular design of MATP, we can independently upgrade each component, making it adaptable for deployment in high-stakes scenarios that require rigorous reasoning validation. Several directions remain for future exploration:

Enhanced Formalization. MATP’s accuracy depends on reliable NL2FOL translation. Current prompt-based methods still show limitations with complex natural language inputs. Future work may incorporate more robust or domain-adapted translation tools. **Open-Domain Knowledge Requirement.** Some mappings (e.g., “Low” to “ $\neg$ High”) require external knowledge not present in the premises. To ensure a strict one-to-one mapping between natural and formal language while handling implicit common knowledge, the model needs to autonomously introduce necessary knowledge (e.g., “low” equals “not high”) either before or during translation.

Improved Evaluation Metrics. Evaluation of extracted FOL typically relies on gold-standard annotations and similarity-based metrics, which often fail to reflect semantic fidelity and logical validity [48]. This underscores the need for more principled and automated evaluation methods.

Reconstructing Reasoning Graphs. Future work may model reasoning steps as nodes and ATP-detected inferences as directed edges, constructing a reasoning graph from LLM-generated reasoning outputs. This structure enables fine-grained verification and helps identify redundant, missing, or conflicting logic beyond simple path existence checks.

Expanded Reasoning Support. Beyond forward entailment, supporting reverse reasoning could broaden applicability.

5.2 Optimization on Response Analysis

To improve efficiency and reduce manual effort—particularly in NL2FOL evaluation, we leverage LLM internal features using entropy and Jensen-Shannon (JS) divergence [31] to identify uncertain or potentially flawed reasoning steps. As demonstrated in Figure 7, when Llama3.1-8B series are evaluated on PROOFWRITER, flawed

reasoning often correlates with higher entropy or JS divergence. Note that the full figure across datasets and LLMs is available in [30].

Based on this, we implement a threshold-based filtering strategy: reasoning outputs with entropy below 7.0 and JS divergence below 0.065 are considered likely correct, allowing us to focus analysis on more uncertain cases. This pre-filtering step effectively reduces the evaluation burden, with correct reasoning coverage rates ranging from 14.7% to 40.5% across datasets and models. While some incorrect responses are mistakenly filtered out (e.g., 8.5% for PROOFWRITER and 6.7% for FOLIO under Llama3.1-8B), other configurations show near-zero false negatives, validating the approach's reliability. Overall, this method improves evaluation efficiency by prioritizing uncertain reasoning steps. Future work will explore tighter integration of LLM internal signals to enable automated filtering and semi-supervised reasoning quality assessment.

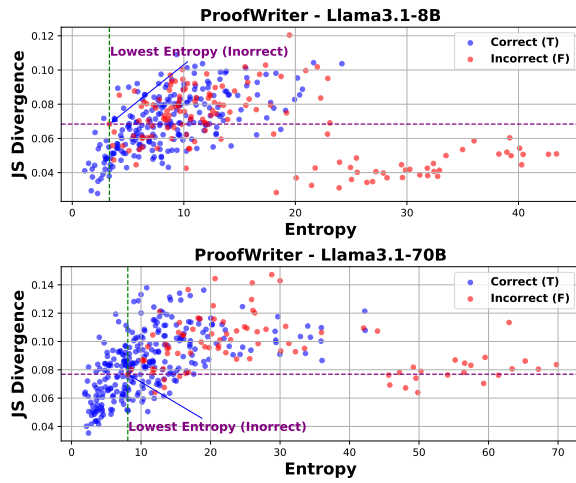


Figure 7: Comparison of entropy and Jensen–Shannon divergence distributions for correct (T) and incorrect (F) reasoning responses for Llama3.1 series on PROOFWRITER.

6 Related Work

Automatically Generating Formal Specifications. LLMs have been increasingly applied to automate formal specification generation and logical translation from natural language. SpecGen [29] adopts a conversational and mutation-based approach to generate program specifications without relying on templates. NL2CTL [61] translates requirements into Computation Tree Logic (CTL) by constructing an NL-CTL dataset through prompt engineering and fine-tuning a T5 model. SAFE [5] targets Rust, integrating data synthesis, fine-tuning, and symbolic verification to automate proof generation with over 70% accuracy. NL2FOL [22] converts natural language into FOL, using SMT solvers for fallacy detection and explanation. NL2SPEC [7] supports interactive translation into temporal logic, allowing iterative refinement to resolve ambiguity.

Enhancing Reasoning Capabilities of LLMs. Enhancing LLM reasoning is critical for high-stakes tasks. Internally, DeepSeek-R1 [14] combines supervised fine-tuning and reinforcement learning to enable reasoning abilities such as chain-of-thought and self-verification. Externally, prompting strategies like Logic-of-Thought [28] and Chain-of-Knowledge [50] improve logical completeness and factual coherence by injecting structured logic or

knowledge into prompts. Tool-augmented methods improve reasoning accuracy by using LLMs as semantic parsers to translating natural language into formal languages, which are subsequently processed by external symbolic tools. LINC [33] converts NL into FOL for logic theorem proving. SatLM [57] translates NL into declarative task specifications for Z3 SMT solver [9], targeting mathematical and logical reasoning tasks. Logic-LM [35] covers four categories of formal languages and corresponding tools, selecting appropriate ones for different types of reasoning tasks. Our work adopts a similar strategy but focuses on evaluating the correctness and logical consistency of LLM responses to reasoning tasks.

Evaluating Reasoning Capabilities of LLMs. Evaluating LLM reasoning has become a key research focus. Prior studies assess model performance across diverse tasks, such as code understanding correctness at both the function [27] and software engineering [46] levels, as well as factual hallucination detection in real-world reasoning scenarios [24]. As reasoning abilities advance, evaluation objectives have expanded beyond overall task correctness to step-by-step reasoning analysis. The survey [23] summarizes four key dimensions for evaluating reasoning steps: *factuality*, *validity*, *coherence*, and *utility*. Despite progress, most methods either rely on labor-intensive gold-standard reasoning chains [12, 39] or probabilistic LLM-based evaluations [53, 59], which undermine logical rigor. ROSCOE [12] proposes several metrics using embedding-based calculations among reasoning steps, reference steps, and problems for evaluation. MR-GSM8K [59] prompts LLMs to check reasoning correctness, outputting structured fields like *Solution Correctness*, *First Error Step*, and *Error Analysis*. ReasonEval [53] fine-tunes LLMs on high-quality labeled data to score *validity* and *redundancy* for each reasoning step. Our work extends this research line by incorporating automated theorem provers to evaluate LLM-generated reasoning steps.

7 Conclusion

In this paper, we presented MATP, an evaluation framework that bridges the gap between informal LLM reasoning and formal verification through automated theorem proving. By systematically converting natural language reasoning into First-Order Logic and applying rigorous verification at each step, MATP provides unprecedented insight into the logical validity of LLM-generated reasoning chains. Our extensive evaluation demonstrates that MATP not only achieves high accuracy in FOL translation and reasoning verification, but also enables fine-grained classification that distinguishes genuine deductive reasoning from superficial pattern matching or accidental correctness. The stark differences revealed between models underscore the critical need for formal verification in high-stakes applications. While challenges remain in handling complex linguistic constructs, MATP establishes a foundational framework for ensuring reliable, verifiable AI reasoning in safety-critical domains where trust through logical soundness is paramount.

Acknowledgement

This work was supported by the National NSF of China (grants No.62302176, 62572013 and 92582102). Author Ling Shi was supported by NTU-DESAY SV Research Program under Grant 2018-0980.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large Language Models for Mathematical Reasoning: Progresses and Challenges. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, Neele Falk, Sara Papi, and Mike Zhang (Eds.). Association for Computational Linguistics, St. Julian's, Malta, 225–237. <https://aclanthology.org/2024.eacl-srw.17/>
- [3] Isabelle Augenstein, Timothy Baldwin, Meeyoung Cha, Tanmoy Chakraborty, Giovanni Luca Ciampaglia, David Corney, Renee DiResta, Emilio Ferrara, Scott Hale, Alon Halevy, Eduard Hovy, Heng Ji, Filippo Menczer, Ruben Miguez, Preslav Nakov, Dietram Scheufele, Shivam Sharma, and Giovanni Zagni. 2024. Factuality challenges in the era of large language models and opportunities for fact-checking. *Nature Machine Intelligence* 6, 8 (2024), 852–863. <https://doi.org/10.1038/s42256-024-00881-z>
- [4] Shiqi Chen, Yiran Zhao, Jinghan Zhang, I-Chun Chern, Siyang Gao, Pengfei Liu, and Junxian He. 2023. FELM: Benchmarking Factuality Evaluation of Large Language Models. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=jSO7Vgole6>
- [5] Tianyu Chen, Shuai Lu, Shan Lu, Yeyun Gong, Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Hao Yu, Nan Duan, Peng Cheng, et al. 2024. Automated proof generation for rust code via self-evolution. *arXiv preprint arXiv:2410.15756* (2024).
- [6] Jan Clusmann, Fiona R. Kolbinger, Hannah Sophie Muti, Zunamys I. Carrero, Jan-Niklas Eckardt, Narmin Ghaffari Laleh, Chiara Maria Lavinia Löffler, Sophie-Caroline Schwarzkopf, Michaela Unger, Gregory P. Veldhuizen, Sophia J. Wagner, and Jakob Nikolas Kather. 2023. The future landscape of large language models in medicine. *Communications Medicine* 3, 1 (2023), 141. <https://doi.org/10.1038/s43856-023-00370-1>
- [7] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. 2023. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *International Conference on Computer Aided Verification*. Springer, 383–396.
- [8] Yu Cui and Cong Zuo. 2025. Practical Reasoning Interruption Attacks on Reasoning Large Language Models. *arXiv preprint arXiv:2505.06643* (2025).
- [9] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [10] Eva Eigner and Thorsten Händler. 2024. Determinants of llm-assisted decision-making. *arXiv preprint arXiv:2402.17385* (2024).
- [11] Esther Gan, Yiran Zhao, Liying Cheng, Mao Yancan, Anirudh Goyal, Kenji Kawaguchi, Min-Yen Kan, and Michael Shieh. 2024. Reasoning Robustness of LLMs to Adversarial Typographical Errors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 10449–10459. <https://doi.org/10.18653/v1/2024.emnlp-main.584>
- [12] Olga Golovneva, Moya Chen, Spencer Poff, Martin Corredor, Luke Zettlemoyer, Maryam Fazel-Zarandi, and Asli Celikyilmaz. 2022. Roscoe: A suite of metrics for scoring step-by-step reasoning. *arXiv preprint arXiv:2212.07919* (2022).
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [15] Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alexander Wardle-Solano, Hannah Szabo, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Ninyong Nan, Junjo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Maciej Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. 2024. FOLIO: Natural Language Reasoning with First-Order Logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 22017–22031. <https://doi.org/10.18653/v1/2024.emnlp-main.1229>
- [16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [17] Jie Huang and Kevin Chen-Chuan Chang. 2023. Towards Reasoning in Large Language Models: A Survey. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 1049–1065. <https://doi.org/10.18653/v1/2023.findings-acl.67>
- [18] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Adam Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [19] Laura Kovács and Andrei Voronkov. 2013. First-Order Theorem Proving and Vampire. In *Computer Aided Verification*, Natasha Sharygina and Helmut Veith (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–35.
- [20] Satyapriya Krishna, Jiaqi Ma, Dylan Z Slack, Asma Ghandeharion, Sameer Singh, and Himabindu Lakkaraju. 2023. Post Hoc Explanations of Language Models Can Improve Language Models. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=3H37XciUEv>
- [21] Jinqi Lai, Wensheng Gan, Jiayang Wu, Zhenlian Qi, and Philip S. Yu. 2024. Large language models in law: A survey. *AI Open* 5 (2024), 181–196. <https://doi.org/10.1016/j.aiopen.2024.09.002>
- [22] Abhinav Lalwani, Lovish Chopra, Christopher Hahn, Caroline Trippel, Zhijing Jin, and Mrinmaya Sachan. 2024. NL2fol: Translating natural language to first-order logic for logical fallacy detection. *arXiv preprint arXiv:2405.02318* (2024).
- [23] Jinu Lee and Julia Hockenmaier. 2025. Evaluating Step-by-step Reasoning Traces: A Survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 1789–1814. <https://doi.org/10.18653/v1/2025.findings-emnlp.94>
- [24] Ningke Li, Yuekang Li, Yi Liu, Ling Shi, Kailong Wang, and Haoyu Wang. 2024. Drowzee: Metamorphic Testing for Fact-Conflicting Hallucination Detection in Large Language Models. 8, OOPSLA2, Article 336 (Oct. 2024), 30 pages. <https://doi.org/10.1145/3689776>
- [25] Yanda Li, Dixuan Wang, Jiaqing Liang, Guochao Jiang, Qianyu He, Yanghua Xiao, and Deqing Yang. 2024. Reason from Fallacy: Enhancing Large Language Models' Logical Reasoning through Logical Fallacy Understanding. In *Findings of the Association for Computational Linguistics: NAACL 2024*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 3053–3066. <https://doi.org/10.18653/v1/2024.findings-naacl.192>
- [26] Yinheng Li, Shaofei Wang, Han Ding, and Hang Chen. 2023. Large Language Models in Finance: A Survey. In *Proceedings of the Fourth ACM International Conference on AI in Finance* (Brooklyn, NY, USA) (ICAIF '23). Association for Computing Machinery, New York, NY, USA, 374–382. <https://doi.org/10.1145/3604237.3626869>
- [27] Ziyu Li and Donghwan Shin. 2024. Mutation-based Consistency Testing for Evaluating the Code Understanding Capability of LLMs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI* (Lisbon, Portugal) (CAIN '24). Association for Computing Machinery, New York, NY, USA, 150–159. <https://doi.org/10.1145/3644815.3644946>
- [28] Tongxuan Liu, Wenjiang Xu, Weizhe Huang, Yuting Zeng, Jiaxing Wang, Xingyu Wang, Hailong Yang, and Jing Li. 2025. Logic-of-Thought: Injecting Logic into Contexts for Full Reasoning in Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 10168–10185. <https://doi.org/10.18653/v1/2025.naacl-long.510>
- [29] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 16–28. <https://doi.org/10.1109/ICSE55347.2025.00129>
- [30] MATP. 2025. <https://github.com/zxyhp/MATP>.
- [31] M.L. Menéndez, J.A. Pardo, L. Pardo, and M.C. Pardo. 1997. The Jensen-Shannon divergence. *Journal of the Franklin Institute* 334, 2 (1997), 307–318. [https://doi.org/10.1016/S0016-0032\(96\)00063-4](https://doi.org/10.1016/S0016-0032(96)00063-4)
- [32] Niels Mündler, Jingxuan He, Slobodan Jenko, and Martin Vechev. 2024. Self-contradictory Hallucinations of Large Language Models: Evaluation, Detection and Mitigation. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=EmQSOiX2f>
- [33] Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. 2023. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 5153–5176. <https://doi.org/10.18653/v1/2023.emnlp-main.313>

- [34] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>. Accessed: 2025-07-13.
- [35] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. LogicLM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 3806–3824. <https://doi.org/10.18653/v1/2023.findings-emnlp.248>
- [36] Amirreza Payandeh, Dan Pluth, Jordan Hosier, Xuesu Xiao, and Vijay K. Gurbani. 2024. How Susceptible Are LLMs to Logical Fallacies?. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). ELRA and ICCL, Torino, Italia, 8276–8286. <https://aclanthology.org/2024.lrec-main.726/>
- [37] Shuofei Qiao, Yixin Ou, Ningyu Zhang, Xiang Chen, Yunzhi Yao, Shumin Deng, Chuanqi Tan, Fei Huang, and Huajun Chen. 2023. Reasoning with Language Model Prompting: A Survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 5368–5393. <https://doi.org/10.18653/v1/2023.acl-long.294>
- [38] Qwen Team. 2025. QwQ-32B: Embracing the Power of Reinforcement Learning. <https://qwenlm.github.io/blog/qwq-32b/>. Accessed: 2025-07-13.
- [39] Abulhair Saparov and He He. 2023. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=qFVBzXxR2V>
- [40] Abulhair Saparov, Richard Yuanzhe Pang, Vishakh Padmakumar, Nitish Joshi, Seyed Mehran Kazemi, Najoung Kim, and He He. 2023. Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples. In *Advances in Neural Information Processing Systems*, Vol. 36. 3083–3105. https://proceedings.neurips.cc/paper_files/paper/2023/file/09425891e393e64b0535194a81ba15b7-Paper-Conference.pdf
- [41] Sergio Servantez, Joe Barrow, Kristian Hammond, and Rajiv Jain. 2024. Chain of Logic: Rule-Based Reasoning with Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2721–2733. <https://doi.org/10.18653/v1/2024.findings-acl.159>
- [42] Parshin Shojaei, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity. <https://ml-site.cdn-apple.com/papers/the-illusion-of-thinking.pdf>
- [43] Geoff Sutcliffe. 2017. The TPTP Problem Library and Associated Infrastructure. *J. Autom. Reason.* 59, 4 (Dec. 2017), 483–502. <https://doi.org/10.1007/s10817-017-9407-7>
- [44] Oyvind Taffjord, Bhavana Dalvi, and Peter Clark. 2021. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 3621–3634. <https://doi.org/10.18653/v1/2021.findings-acl.317>
- [45] Derek Tam, Anisha Mascarenhas, Shiyue Zhang, Sarah Kwan, Mohit Bansal, and Colin Raffel. 2023. Evaluating the Factual Consistency of Large Language Models Through News Summarization. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 5220–5255. <https://doi.org/10.18653/v1/2023.findings-acl.322>
- [46] Minaoar Hossain Tanzil, Junaed Younus Khan, and Gias Uddin. 2024. ChatGPT Incorrectness Detection in Software Reviews. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 180, 12 pages. <https://doi.org/10.1145/3597503.3639194>
- [47] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>. Accessed: 2025-07-13.
- [48] Ramya Keerthy Thatikonda, Wray Buntine, and Ehsan Shareghi. 2025. Assessing the Sensitivity and Alignment of FOL Closeness Metrics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 16775–16785. <https://doi.org/10.18653/v1/2025.findings-emnlp.910>
- [49] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient Large Language Models: A Survey. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=bsCCJHbO8A> Survey Certification.
- [50] Jianing Wang, Qiushi Sun, Xiang Li, and Ming Gao. 2024. Boosting Language Models Reasoning with Chain-of-Knowledge Prompting. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 4958–4981. <https://doi.org/10.18653/v1/2024.acl-long.271>
- [51] Xuezhong Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=1PL1NIMMrw>
- [52] Junda Wu, Tong Yu, Xiang Chen, Haoliang Wang, Ryan Rossi, Sungchul Kim, Anup Rao, and Julian McAuley. 2024. DeCoT: Debiasing Chain-of-Thought for Knowledge-Intensive Tasks in Large Language Models via Causal Intervention. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 14073–14087. <https://doi.org/10.18653/v1/2024.acl-long.758>
- [53] Shijie Xia, Xuefeng Li, Yixin Liu, Tongshuang Wu, and Pengfei Liu. 2025. Evaluating mathematical reasoning beyond accuracy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27723–27730.
- [54] Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-RL: Unleashing LLM reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768* (2025).
- [55] Sherry Yang, Ofir Nachum, Yilun Du, Jason Wei, Pieter Abbeel, and Dale Schuurmans. 2023. Foundation models for decision making: Problems, methods, and opportunities. *arXiv preprint arXiv:2303.04129* (2023).
- [56] Yuan Yang, Siheng Xiong, Ali Payani, Ehsan Shareghi, and Faramarz Fekri. 2024. Harnessing the Power of Large Language Models for Natural Language to First-Order Logic Translation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 6942–6959. <https://doi.org/10.18653/v1/2024.acl-long.375>
- [57] Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 45548–45580. https://proceedings.neurips.cc/paper_files/paper/2023/file/8e9c7d4a48bdac81a58f983a64aaf42b-Paper-Conference.pdf
- [58] Junqi Yin, Sajal Dash, Feiyi Wang, and Mallikarjun Shankar. 2023. FORGE: Pre-Training Open Foundation Models for Science. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Denver, CO, USA) (SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 81, 13 pages. <https://doi.org/10.1145/3581784.3613215>
- [59] Zhongshen Zeng, Pengguang Chen, Shu Liu, Haiyun Jiang, and Jiaya Jia. 2023. MR-GSM8K: A Meta-Reasoning Benchmark for Large Language Model Evaluation. *CoRR abs/2312.17080* (2023). <https://doi.org/10.48550/ARXIV.2312.17080> arXiv:2312.17080
- [60] Yu Zhang, Xiuxi Chen, Bowen Jin, Sheng Wang, Shuiwang Ji, Wei Wang, and Jiawei Han. 2024. A Comprehensive Survey of Scientific Large Language Models and Their Applications in Scientific Discovery. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8783–8817. <https://doi.org/10.18653/v1/2024.emnlp-main.498>
- [61] Mengyan Zhao, Ran Tao, Yanhong Huang, Jianqi Shi, Shengchao Qin, and Yang Yang. 2024. NL2CTL: Automatic Generation of Formal Requirements Specifications via Large Language Models. In *Formal Methods and Software Engineering*, Kazuhiro Ogata, Dominique Mery, Meng Sun, and Shaoying Liu (Eds.). Springer Nature Singapore, Singapore, 1–17.